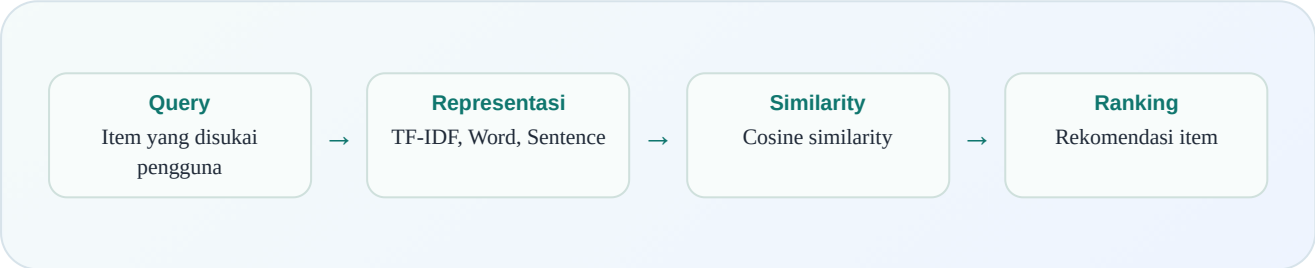
LAPORAN KAJIAN TUGAS KULIAH

Content-Based Filtering dengan Word Embedding dan Sentence Embedding

Kajian konsep, perbandingan representasi teks, dan implementasi sederhana untuk sistem rekomendasi berbasis deskripsi item



DISUSUN OLEH
Najib Robbani

BIDANG
Sistem Rekomendasi dan NLP

TANGGAL
Oktober 2026

REPOSITORI
github.com/najibrobbani74/college-task

Abstrak

Content-based filtering (CBF) merekomendasikan item berdasarkan kemiripan antara profil pengguna dan fitur item. Pada domain teks, kualitas rekomendasi sangat ditentukan oleh cara deskripsi item direpresentasikan. Kajian ini membahas dua pendekatan representasi yang banyak digunakan: **word embedding**, yang memetakan setiap kata ke vektor padat berdasarkan konteks pemakaiannya, dan **sentence embedding**, yang memetakan satu kalimat utuh ke vektor yang menangkap makna dan komposisinya. Kajian dilengkapi implementasi sederhana pada 12 deskripsi film. Word embedding dibangun dari matriks co-occurrence, PPMI, dan SVD; sentence embedding dihitung dengan mean pooling dan dibandingkan dengan Sentence-BERT. Hasil demonstrasi menunjukkan bahwa ketiga metode (TF-IDF, word embedding, dan Sentence-BERT) sama-sama menempatkan *The Martian* sebagai rekomendasi teratas untuk query *Interstellar*, tetapi urutan berikutnya berbeda. TF-IDF dan mean pooling masih dapat tertipu oleh kata yang sama pada konteks berbeda, sedangkan Sentence-BERT menghasilkan peringkat yang lebih koheren secara semantik. Kajian ini menyimpulkan bahwa pemilihan representasi adalah trade-off antara kualitas semantik, biaya komputasi, dan kemudahan interpretasi.

Kata kunci: content-based filtering, word embedding, sentence embedding, Word2Vec, Sentence-BERT, cosine similarity

Daftar Isi

1 Pendahuluan

2 Content-Based Filtering

3 Word Embedding

4 Sentence Embedding

5 Perbandingan dan Pembahasan

6 Kesimpulan

— Referensi

A Lampiran A: Struktur dan Cara Menjalankan Program

B Lampiran B: Ringkasan Output Program

Cara membaca laporan ini. Bagian 2 menjelaskan kerangka CBF dan baseline TF-IDF. Bagian 3 dan 4 membahas word embedding dan sentence embedding, masing-masing dengan contoh sederhana. Bagian 5 membandingkan hasil, dan Bagian 6 merangkum. Program pendukung serta demo interaktif dapat diakses melalui repositori yang tercantum pada halaman sampul.

1. Pendahuluan

1.1 Latar belakang

Jumlah konten digital yang tersedia bagi pengguna terus bertambah. Katalog film, buku, berita, produk, dan materi pembelajaran jauh melampaui kemampuan pengguna untuk memeriksanya satu per satu. Sistem rekomendasi muncul sebagai solusi untuk mengurangi beban tersebut: ia memilih sejumlah kecil item yang diperkirakan relevan, lalu menyusunnya dalam urutan yang berguna.

Secara umum, sistem rekomendasi dapat dibagi menjadi dua keluarga besar. **Collaborative filtering** memanfaatkan pola interaksi banyak pengguna, misalnya "pengguna yang menyukai A juga menyukai B". **Content-based filtering** (CBF) tidak membutuhkan pola pengguna lain. Ia membangun profil pengguna dari item yang disukai, lalu membandingkan profil tersebut dengan fitur setiap item. Karena itu, CBF relatif kuat ketika item baru belum memiliki interaksi, tetapi lemah ketika pengguna baru belum memiliki profil.

Pada CBF, fitur item dapat berupa metadata terstruktur (genre, tahun, penulis) atau teks bebas (sinopsis, deskripsi produk, isi berita). Teks menyimpan banyak informasi, tetapi tidak langsung dapat dibandingkan secara numerik. Di sinilah representasi teks berperan. Pendekatan klasik seperti **TF-IDF** mengubah teks menjadi vektor renggang berdasarkan frekuensi kata. Pendekatan ini cepat dan mudah dijelaskan, tetapi hanya mengenali kata yang sama persis. Ia tidak tahu bahwa "astronaut" dan "spaceman" bermakna dekat, atau bahwa kata "space" pada "space exploration" berbeda maknanya dari "space" pada "space ranger".

Word embedding dan **sentence embedding** menawarkan jalan keluar dari keterbatasan leksikal tersebut. Word embedding memberi setiap kata sebuah vektor padat yang dipelajari dari konteks; kata-kata yang sering muncul dalam konteks serupa akan berdekatan. Sentence embedding melangkah lebih jauh: ia memetakan satu kalimat atau paragraf menjadi satu vektor, sehingga urutan, konteks, dan komposisi kata ikut dipertimbangkan. Kedua pendekatan ini menjadi fondasi banyak sistem pencarian semantik dan rekomendasi modern.

1.2 Rumusan masalah

Kajian ini menjawab empat pertanyaan berikut.

1. Bagaimana content-based filtering bekerja dan mengapa representasi item menjadi kunci?
2. Bagaimana word embedding dan sentence embedding merepresentasikan teks?
3. Bagaimana kedua pendekatan digunakan dalam CBF, serta apa kelebihan dan keterbatasannya?
4. Apa yang ditunjukkan oleh contoh sederhana ketika ketiga representasi dibandingkan pada data yang sama?

1.3 Tujuan

- Menjelaskan konsep dasar CBF, TF-IDF, dan cosine similarity.
- Menjelaskan cara kerja word embedding (Word2Vec, GloVe, FastText, dan PPMI+SVD) serta penggunaannya dalam CBF.
- Menjelaskan sentence embedding (mean pooling, Doc2Vec, Universal Sentence Encoder, dan Sentence-BERT) serta penggunaannya.
- Menyajikan contoh sederhana yang dapat dijalankan ulang dan membandingkan hasil ketiga representasi.
- Merumuskan rekomendasi praktis mengenai kapan masing-masing pendekatan sebaiknya digunakan.

1.4 Batasan

Kajian ini berfokus pada CBF berbasis teks dan tidak membahas collaborative filtering secara mendalam. Eksperimen menggunakan 12 deskripsi film berbahasa Inggris dan bukan merupakan benchmark produksi. Word embedding dilatih dari korpus kecil dengan PPMI+SVD agar prosesnya transparan; ini bukan Word2Vec yang dilatih pada miliaran kata. Sentence-BERT menggunakan model `all-MiniLM-L6-v2` yang sudah dilatih. Angka-angka pada laporan bersifat ilustratif dan dapat berubah pada data atau model lain.

1.5 Metode

Metode yang digunakan adalah kajian pustaka yang dilengkapi eksperimen kecil. Kajian pustaka mencakup buku sistem rekomendasi, makalah Word2Vec, GloVe, FastText, Doc2Vec, Sentence-BERT, dan literatur evaluasi sistem rekomendasi. Eksperimen dijalankan melalui program Python `cbf_embeddings.py` yang menghasilkan tiga representasi untuk data yang sama: TF-IDF, word embedding dengan mean pooling, dan Sentence-BERT. Hasilnya ditampilkan pada halaman demo interaktif dan dirangkum di Bagian 5.

2. Content-Based Filtering

2.1 Definisi dan intuisi

Content-based filtering adalah pendekatan yang merekomendasikan item berdasarkan kemiripan antara **profil pengguna** dan **profil item**. Profil item dibangun dari fitur item; profil pengguna dibangun dari item yang pernah disukai, dibeli, dibaca, atau diberi rating tinggi. Skor relevansi biasanya berupa fungsi kemiripan:

> **Rumus.** score(u, i) = similarity(profile(u), features(i))
>
> profile(u): representasi preferensi pengguna; features(i): representasi fitur item; similarity: fungsi seperti cosine, Euclidean, atau Jaccard.

Intuisinya sederhana: jika pengguna menyukai film tentang astronot yang berjuang di luar angkasa, sistem mencari item lain yang memiliki fitur serupa, misalnya deskripsi tentang astronot, planet, dan bertahan hidup. Berbeda dari collaborative filtering, CBF tidak memerlukan data pengguna lain; ia hanya membutuhkan fitur item dan preferensi pengguna.

2.2 Komponen sistem

```
Item profile  →  User profile  →  Scoring (cosine)  →  Ranking & feedback
```

Komponen pertama adalah **item representation**. Fitur dapat berupa genre, sutradara, tahun, kata kunci, atau deskripsi lengkap. Pada sistem berbasis teks, deskripsi perlu diubah menjadi vektor. Komponen kedua adalah **user profile**. Profil dapat berupa vektor rata-rata dari item yang disukai, vektor berbobot rating, atau model yang dilatih dari umpan balik. Komponen ketiga adalah **similarity function**. Cosine similarity paling sering dipakai karena tidak terpengaruh panjang vektor dan mudah dihitung. Komponen keempat adalah **ranking** dan **feedback loop**: item dengan skor tertinggi ditampilkan, lalu interaksi pengguna memperbarui profil.

2.3 Baseline TF-IDF dan cosine similarity

TF-IDF adalah baseline klasik yang kuat untuk teks. Bobot sebuah kata meningkat jika kata itu sering muncul di dokumen tertentu, tetapi menurun jika kata itu muncul di banyak dokumen. Secara ringkas:

> **Rumus.** tf-idf(t, d) = tf(t, d) × idf(t), dengan idf(t) = ln((1 + N) / (1 + df(t))) + 1
>
> tf(t, d): frekuensi kata t pada dokumen d; df(t): jumlah dokumen yang memuat t; N: jumlah dokumen. Vektor kemudian dinormalisasi L2.

Setelah setiap dokumen menjadi vektor TF-IDF, kemiripan dua dokumen diukur dengan cosine similarity:

> **Rumus.** cos(q, d) = (q · d) / (||q|| × ||d||)
>
> q: vektor query atau profil pengguna; d: vektor item. Nilai 1 berarti searah, 0 berarti tidak memiliki komponen bersama, dan -1 berarti berlawanan arah.

Sebagai contoh, query *Interstellar* memiliki deskripsi "A team of astronauts travels through a wormhole in space to find a new home for humanity." Kata *astronaut*, *space*, dan *wormhole* mendapat bobot tinggi karena relatif jarang muncul di dokumen lain. Item *The Martian* berbagi kata *astronaut* dan *Mars*, sehingga skornya tinggi. Item *Toy*

Story juga berbagi kata *space* dan *new*, sehingga bisa ikut naik meskipun konteksnya berbeda. Inilah salah satu kelemahan TF-IDF: ia tidak memahami makna, hanya keberadaan kata.

2.4 Kelebihan dan keterbatasan TF-IDF

Kelebihan	Keterbatasan
Cepat, ringan, dan tidak memerlukan pelatihan model.	Tidak menangkap sinonim dan parafrasa.
Mudah dijelaskan: bobot kata dapat diperiksa langsung.	Gagal membedakan polisemi, misalnya "space" pada konteks berbeda.
Kuat untuk pencarian kata kunci dan domain dengan istilah teknis.	Tidak mempertimbangkan urutan kata dan negasi.
Menjadi baseline yang baik sebelum mencoba model yang lebih mahal.	Vektor renggang dan rapuh pada teks pendek.

> **Catatan.** Keterbatasan TF-IDF bukan alasan untuk membuangnya. TF-IDF tetap berguna sebagai filter awal atau baseline. Namun, untuk menangkap makna, kita membutuhkan representasi yang lebih kaya: word embedding dan sentence embedding.

3. Word Embedding

3.1 Definisi dan hipotesis distribusional

Word embedding adalah representasi kata sebagai vektor bilangan real berdimensi tetap, misalnya 50, 100, 300, atau 384 dimensi. Berbeda dari one-hot encoding yang panjang dan renggang, word embedding bersifat padat: hampir semua elemennya bukan nol. Vektor ini dipelajari dari data teks, bukan ditentukan manual.

Dasar filosofisnya adalah **hipotesis distribusional**: kata yang muncul dalam konteks serupa cenderung memiliki makna serupa. Firth merumuskannya secara terkenal sebagai "you shall know a word by the company it keeps". Jika "astronaut" sering muncul dekat "space", "Mars", dan "survive", maka vektornya akan berdekatan dengan kata-kata tersebut.

```
Korpus teks  →  Konteks kata  →  Co-occurrence / prediksi  →  Vektor padat
```

3.2 Word2Vec: CBOW dan Skip-gram

Word2Vec memperkenalkan dua arsitektur yang efisien untuk mempelajari word embedding dari korpus besar. **CBOW** (Continuous Bag of Words) memprediksi kata pusat berdasarkan kata-kata di sekitarnya. **Skip-gram** melakukan kebalikannya: memprediksi kata-kata konteks berdasarkan kata pusat. Skip-gram biasanya lebih baik untuk kata yang jarang muncul, sedangkan CBOW lebih cepat.

> **Rumus.** Skip-gram: maksimalkan Σ log P(w_{t+j} | w_t), untuk -m ≤ j ≤ m, j ≠ 0
>
> w_t: kata pusat; w_{t+j}: kata konteks; m: ukuran jendela. Probabilitas dihitung dengan softmax, lalu didekati dengan negative sampling agar pelatihan tetap efisien pada kosakata besar.

Setelah pelatihan, setiap kata memiliki vektor. Cosine similarity antara dua vektor menjadi ukuran kedekatan makna. Word2Vec juga menunjukkan bahwa operasi vektor dapat menangkap relasi analogi, misalnya *king − man + woman ≈ queen* pada model yang dilatih dengan korpus besar. Sifat ini tidak dijamin pada korpus kecil.

3.3 GloVe dan FastText

GloVe (Global Vectors) menggunakan matriks co-occurrence global. Ia tidak memprediksi konteks per jendela seperti Word2Vec, tetapi memfaktorkan matriks co-occurrence dengan fungsi objektif berbobot. Rasio probabilitas co-occurrence, misalnya antara "ice" dan "steam", digunakan untuk menangkap dimensi makna seperti solid vs gas.

FastText memperluas Word2Vec dengan subword n-gram. Setiap kata dipecah menjadi potongan huruf, sehingga kata yang tidak pernah muncul saat pelatihan masih dapat diwakili dari subword-nya. Pendekatan ini sangat berguna untuk bahasa dengan morfologi kaya, termasuk bahasa Indonesia, dan untuk menangani kata yang salah ketik.

3.4 Word embedding statis vs kontekstual

Word2Vec, GloVe, dan FastText menghasilkan **embedding statis**: satu kata memiliki satu vektor, terlepas dari konteks kalimat. Model kontekstual seperti ELMo dan BERT menghasilkan vektor yang berubah sesuai kalimat. Kata "bank" pada "bank account" dan "river bank" akan memiliki vektor berbeda. Model kontekstual lebih kuat, tetapi lebih mahal secara komputasi. Pada kajian ini, istilah "word embedding" mengacu pada embedding statis, sedangkan model kontekstual dibahas sebagai jembatan menuju sentence embedding.

3.5 Contoh sederhana: PPMI + SVD

Untuk menunjukkan prinsipnya tanpa bergantung pada model besar, program demo membangun word embedding dari 12 deskripsi film. Langkahnya:

1. Tokenisasi, penghapusan stopword, dan normalisasi bentuk jamak.
2. Penghitungan co-occurrence dalam jendela dua kata.
3. Konversi counts menjadi Positive Pointwise Mutual Information (PPMI).
4. Reduksi dimensi dengan Truncated SVD menjadi 48 dimensi.
5. Normalisasi L2 setiap vektor.

PPMI memberi bobot tinggi pada pasangan kata yang muncul lebih sering dari yang diharapkan secara acak, dan nol pada pasangan yang muncul lebih jarang. SVD mengambil pola utama dari matriks renggang tersebut. Hasilnya adalah vektor padat yang menangkap kemiripan konteks. Pada korpus demo diperoleh 92 kata kosakata, matriks co-occurrence 92 × 92, 366 sel PPMI tak nol, dan explained variance SVD sebesar 89,20%.

Kata query	Tetangga terdekat (cosine)
space	shuttle (0,56), destroy (0,42), ranger (0,34), orbit (0,34), travel (0,32), become (0,32), new (0,31), home (0,31)
astronaut	survive (0,45), fight (0,45), mars (0,43), stranded (0,43), travel (0,42), wormhole (0,41), debris (0,36), team (0,30)
love	fall (0,67), young (0,58), test (0,38), artist (0,37), los (0,35), doomed (0,33), aspiring (0,28), war (0,25)
crime	patriarch (0,75), aging (0,52), control (0,49), family (0,42), reluctant (0,36), navigate (0,20), expectation (0,20), transfer (0,18)
dream	los (0,67), sharing (0,40), plant (0,37), pursuing (0,36), angeles (0,33), steal (0,32), through (0,32), thief (0,29)
family	aging (0,66), reluctant (0,42), crime (0,42), young (0,41), expectation (0,39), transfer (0,38), navigate (0,37), romance (0,29)

Tabel 1. Tetangga terdekat word embedding pada korpus demo. Skor adalah cosine similarity; nilai lebih tinggi berarti konteks pemakaian lebih mirip.

Hasil ini menunjukkan pola yang masuk akal. *astronaut* berdekatan dengan *survive*, *Mars*, *stranded*, dan *wormhole*. *love* berdekatan dengan *fall*, *young*, dan *artist*. *crime* berdekatan dengan *patriarch*, *family*, dan *reluctant*. Namun, *space* juga berdekatan dengan *ranger* karena *Toy Story* memuat frasa "space ranger". Ini contoh kecil polisemi yang tidak dapat diselesaikan oleh embedding statis.

3.6 Menggunakan word embedding dalam CBF

Untuk membandingkan dokumen, vektor kata perlu digabungkan menjadi vektor dokumen. Cara paling sederhana adalah mean pooling:

> **Rumus.** $d = (1 / |T|) \times \Sigma_{w \in T} v_w$
>
> T: himpunan kata dalam dokumen setelah stopword removal; v_w: vektor kata w. Vektor d kemudian dinormalisasi L2.

Alternatif yang lebih baik adalah pembobotan TF-IDF: kata yang penting diberi bobot lebih besar, kata umum diberi bobot lebih kecil. Setelah vektor dokumen terbentuk, CBF cukup menghitung cosine similarity antara vektor query dan vektor setiap item.

```
def mean_sentence_vector(text, word_vectors):
    words = [w for w in tokenize(text) if w in word_vectors]
    if not words:
        return None
    vector = np.mean([word_vectors[w] for w in words], axis=0)
    return vector / np.linalg.norm(vector)
```

Pada demo, query *Interstellar* dan kandidat *The Martian* memiliki dot product 0,1790, norma query 0,5397, norma kandidat 0,6348, sehingga cosine similarity-nya 0,5225. Angka ini lebih tinggi daripada cosine TF-IDF untuk pasangan yang sama (0,1743), karena word embedding ikut memanfaatkan kedekatan makna kata, bukan hanya kata yang sama persis.

3.7 Kelebihan dan keterbatasan

Kelebihan	Keterbatasan
Menangkap kemiripan makna dan variasi kata.	Mean pooling menghilangkan urutan kata dan negasi.
Vektor padat dan relatif ringan setelah dilatih.	Embedding statis memberi satu vektor untuk semua makna kata.
Dapat digunakan untuk teks pendek dan query singkat.	Kata di luar kosakata (OOV) tidak memiliki vektor.
Mudah diintegrasikan ke CBF melalui mean pooling.	Kualitas sangat bergantung pada korpus dan domain pelatihan.

4. Sentence Embedding

4.1 Dari kata ke kalimat

Word embedding memberi representasi per kata. Namun, rekomendasi sering bekerja pada tingkat kalimat atau paragraf: sinopsis film, deskripsi produk, abstrak artikel. Jika kita hanya merata-ratakan vektor kata, kita kehilangan informasi urutan dan komposisi. Kalimat "anjing menggigit orang" dan "orang menggigit anjing" memiliki kata yang sama, tetapi maknanya berbeda. Sentence embedding bertujuan memetakan seluruh kalimat menjadi satu vektor yang mempertahankan makna tersebut.

```
Token / kata  →  Encoder kontekstual  →  Pooling  →  Vektor kalimat
```

4.2 Pendekatan sederhana

Ada beberapa cara sederhana untuk membentuk sentence embedding. **Mean pooling** merata-ratakan vektor kata. **TF-IDF weighted mean** memberi bobot lebih besar pada kata yang penting. **SIF** (Smooth Inverse Frequency) memberi bobot yang berbanding terbalik dengan frekuensi kata, lalu menghapus komponen utama pertama untuk mengurangi pengaruh kata umum. **Doc2Vec** memperlakukan dokumen sebagai vektor yang dilatih bersama kata-kata di dalamnya, dengan arsitektur PV-DM atau PV-DBOW. **LSA** menggunakan SVD pada matriks term-document untuk menghasilkan vektor dokumen.

Pendekatan-pendekatan ini murah dan sering menjadi baseline yang kuat, tetapi belum tentu menangkap komposisi dan konteks dengan baik. Model neural seperti Universal Sentence Encoder dan Sentence-BERT dilatih khusus untuk menghasilkan vektor kalimat yang bermakna.

4.3 Sentence-BERT

BERT menghasilkan embedding kontekstual per token, tetapi vektor `[CLS]` mentah tidak selalu cocok untuk cosine similarity tanpa fine-tuning. Sentence-BERT (SBERT) mengatasi masalah ini dengan **siamese network**: dua kalimat dimasukkan ke encoder yang sama, lalu vektor hasil pooling dibandingkan menggunakan cosine. Model dilatih dengan pasangan kalimat dan objective contrastive/triplet, sehingga kalimat yang mirip maknanya didekatkan dan yang berbeda dijauhkan.

> **Rumus.** s = cos(pooling(BERT(sentence_1)), pooling(BERT(sentence_2)))
>
> Pooling biasanya mean pooling. Vektor dinormalisasi L2, sehingga cosine sama dengan dot product. SBERT jauh lebih cepat untuk pencarian kemiripan karena vektor kalimat dapat dihitung sekali dan disimpan.

Model yang digunakan pada demo adalah `all-MiniLM-L6-v2`, menghasilkan vektor 384 dimensi. Model ini kecil, cepat, dan cukup baik untuk kemiripan semantik umum. Pada data demo, seluruh 12 deskripsi film diubah menjadi vektor, lalu cosine similarity dihitung terhadap query.

Peringkat	Item	Skor SBERT	Genre
1	The Martian	0,4770	Sci-Fi, Adventure, Drama
2	Gravity	0,3277	Sci-Fi, Thriller, Drama
3	Arrival	0,3219	Sci-Fi, Drama, Mystery
4	La La Land	0,1371	Romance, Drama, Music
5	Toy Story	0,1088	Animation, Adventure, Comedy

Tabel 2. Rekomendasi Sentence-BERT untuk query *Interstellar*.

Peringkat ini lebih koheren secara semantik daripada baseline leksikal. *The Martian* dan *Gravity* sama-sama tentang astronot dan bertahan hidup di luar angkasa. *Arrival* juga relevan karena berbicara tentang alien, bahasa, dan kontak pertama. Sebaliknya, *Toy Story* turun ke peringkat kelima meskipun deskripsinya memuat kata "space", karena model memahami bahwa "space ranger" adalah mainan, bukan perjalanan antariksa.

4.4 Menggunakan sentence embedding dalam CBF

Alur penggunaannya mirip dengan word embedding, tetapi encoder bekerja pada tingkat kalimat. Deskripsi setiap item diubah menjadi vektor dan disimpan. Profil pengguna dapat berupa rata-rata vektor item yang disukai. Skor rekomendasi dihitung dengan cosine similarity.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
item_vectors = model.encode(
    descriptions,
    normalize_embeddings=True,
    show_progress_bar=False,
)

# user_vector: rata-rata vektor item yang disukai, sudah dinormalisasi
scores = item_vectors @ user_vector
```

Untuk katalog besar, pencarian tetangga terdekat secara brute force bisa menjadi lambat. Sistem produksi biasanya memakai approximate nearest neighbor seperti FAISS, Annoy, HNSW, atau ekstensi vektor pada basis data (misalnya pgvector). Vektor juga perlu diperbarui ketika model atau data berubah, karena embedding dari model berbeda tidak selalu sebanding.

4.5 Kelebihan dan keterbatasan

Kelebihan	Keterbatasan
Menangkap makna kalimat, bukan hanya kata.	Model lebih besar dan lebih lambat daripada TF-IDF.
Lebih tahan terhadap sinonim, parafrasa, dan urutan kata.	Kurang interpretable: sulit menjelaskan dimensi tertentu.
Tersedia model multibahasa, termasuk untuk bahasa Indonesia.	Kualitas turun jika domain data jauh dari data pelatihan.
Cocok untuk pencarian semantik dan rekomendasi teks panjang.	Dapat mewarisi bias dari korpus pelatihan.

5. Perbandingan dan Pembahasan

5.1 Perbandingan konseptual

Aspek	TF-IDF	Word Embedding + mean pooling	Sentence-BERT
Representasi	Vektor renggang berbasis frekuensi kata	Vektor padat; rata-rata vektor kata	Vektor padat; encoder transformer + pooling
Kemiripan makna	Rendah; bergantung kata yang sama	Sedang; menangkap kedekatan kata	Tinggi; menangkap makna kalimat
Urutan dan konteks	Tidak dipertimbangkan	Tidak dipertimbangkan	Dipertimbangkan
Kata baru (OOV)	Diabaikan	Diabaikan, kecuali model subword	Umumnya ditangani tokenizer subword
Pelatihan	Tidak perlu	Perlu korpus untuk melatih vektor	Model pretrained; fine-tuning opsional
Biaya inferensi	Sangat rendah	Rendah	Lebih tinggi; perlu GPU untuk skala besar
Interpretabilitas	Tinggi	Sedang	Rendah
Kebutuhan data	Kecil	Sedang	Besar untuk pelatihan; pretrained siap pakai

Tabel 3. Perbandingan konseptual tiga representasi teks untuk CBF.

5.2 Hasil eksperimen pada query Interstellar

Peringkat	TF-IDF	Word Embedding	Sentence-BERT
1	The Martian (0,174)	The Martian (0,523)	The Martian (0,477)
2	Gravity (0,142)	Gravity (0,486)	Gravity (0,328)
3	Toy Story (0,141)	Toy Story (0,369)	Arrival (0,322)
4	Inception (0,079)	Inception (0,223)	La La Land (0,137)
5	Titanic (0,000)	La La Land (0,037)	Toy Story (0,109)

Tabel 4. Top-5 rekomendasi untuk query *Interstellar* pada tiga representasi. Skor adalah cosine similarity; skala antar-metode tidak selalu sebanding.

Perbandingan skor untuk *The Martian* dan *Toy Story* memperlihatkan pola yang menarik. Pada TF-IDF, *The Martian* hanya sedikit di atas *Toy Story* (0,174 vs 0,141). Pada word embedding, jaraknya melebar (0,523 vs 0,369). Pada Sentence-BERT, *Toy Story* turun jauh (0,109) dan *Arrival* naik ke peringkat ketiga (0,322). Ini menunjukkan bahwa representasi yang lebih semantik tidak hanya mengubah skor, tetapi juga mengubah komposisi rekomendasi.

5.3 Analisis hasil

Ketiga metode sepakat pada peringkat pertama: *The Martian* adalah rekomendasi paling relevan untuk *Interstellar*. Keduanya berbagi tema astronot, luar angkasa, dan bertahan hidup. Namun, urutan berikutnya memperlihatkan perbedaan yang penting.

TF-IDF menempatkan *Toy Story* di peringkat ketiga karena deskripsinya memuat kata "space" dan "new". Kata "space" memang muncul di kedua deskripsi, tetapi konteksnya berbeda: "space exploration" versus "space ranger". TF-IDF tidak memiliki cara untuk membedakannya. Word embedding dengan mean pooling juga menempatkan *Toy Story* di peringkat ketiga, meskipun skornya lebih tinggi. Rata-rata vektor kata masih membawa komponen "space", dan komponen itu bertemu dengan vektor "space" pada query. Ini menunjukkan bahwa mean pooling belum cukup untuk memahami komposisi kalimat.

Sentence-BERT menempatkan *Arrival* di peringkat ketiga, bukan *Toy Story*. *Arrival* memang tidak berbagi banyak kata dengan *Interstellar*, tetapi secara semantik ia dekat: alien, bahasa, Bumi, dan upaya mencegah perang. Model memahami bahwa *Arrival* adalah fiksi ilmiah tentang kontak pertama, bukan mainan yang kebetulan memakai kata "space". *Toy Story* turun ke peringkat kelima, yang lebih masuk akal untuk pengguna yang menyukai *Interstellar*.

5.4 Kapan memakai pendekatan mana?

Kebutuhan	Pilihan yang disarankan
Baseline cepat, data kecil, harus mudah dijelaskan	TF-IDF + cosine; tambahkan metadata genre.
Butuh generalisasi makna kata dengan biaya rendah	Word embedding pretrained + mean pooling atau TF-IDF weighted mean.
Kualitas semantik tinggi, teks panjang, sumber daya memadai	Sentence embedding (Sentence-BERT atau model multibahasa).
Produksi dengan batasan kata kunci dan latensi	Hybrid: filter leksikal + skor semantik + learning-to-rank.

Tabel 5. Panduan praktis pemilihan representasi.

Pada praktiknya, pilihan tidak harus tunggal. Sistem produksi sering menggabungkan beberapa sinyal: genre, popularitas, kata kunci, dan skor semantik. Hybrid ranking dapat menutup kelemahan masing-masing metode. Misalnya, TF-IDF atau filter metadata memastikan kecocokan kata kunci yang wajib, sedangkan sentence embedding menangani parafrasa dan sinonim.

5.5 Evaluasi

Demo ini hanya menunjukkan peringkat secara kualitatif. Sistem nyata perlu dievaluasi dengan metrik kuantitatif. Untuk evaluasi offline, metrik yang umum adalah Precision@k, Recall@k, F1@k, Mean Average Precision (MAP), Mean Reciprocal Rank (MRR), dan Normalized Discounted Cumulative Gain (NDCG). Untuk evaluasi online, dapat digunakan click-through rate, conversion rate, dwell time, dan survei kepuasan pengguna. Evaluasi juga perlu memperhatikan keragaman (diversity), kebaruan (novelty), dan keadilan (fairness), bukan hanya relevansi.

Ground truth untuk CBF tidak selalu tersedia. Salah satu pendekatan adalah menyusun data uji dengan item yang benar-benar disukai pengguna, lalu mengukur apakah item tersebut muncul di top-k. Pendekatan lain adalah human evaluation: responden menilai relevansi rekomendasi. Untuk tugas kuliah ini, evaluasi kualitatif pada contoh terbatas sudah cukup untuk menunjukkan perbedaan perilaku ketiga representasi.

5.6 Tantangan dan keterbatasan

- **Cold start pengguna.** CBF tetap kesulitan jika pengguna belum memiliki item yang disukai.
- **Over-specialization.** Rekomendasi bisa terlalu sempit dan mengurangi keragaman.
- **Filter bubble.** Pengguna terus menerima item yang mirip dengan preferensi lama.
- **Bias dan keadilan.** Embedding dapat mewarisi bias gender, ras, atau budaya dari korpus.
- **Domain shift.** Model yang dilatih pada berita belum tentu cocok untuk sinopsis film atau produk.
- **Biaya dan latensi.** Sentence embedding membutuhkan lebih banyak sumber daya daripada TF-IDF.
- **Explainability.** Sulit menjelaskan mengapa dua vektor dianggap mirip tanpa alat tambahan.

Keterbatasan eksperimen ini juga perlu dicatat: data hanya 12 film, semuanya berbahasa Inggris, tanpa data interaksi pengguna, dan word embedding dilatih dari korpus yang sangat kecil. Hasilnya tidak dapat digeneralisasi ke produksi. Meskipun demikian, contoh ini cukup untuk memperlihatkan perbedaan konsep dan perilaku ketiga representasi.

6. Kesimpulan

Content-based filtering merekomendasikan item berdasarkan kemiripan antara profil pengguna dan fitur item. Pada domain teks, kualitas CBF sangat ditentukan oleh representasi yang dipakai. TF-IDF tetap menjadi baseline yang cepat, murah, dan mudah dijelaskan, tetapi hanya memahami kata yang sama persis. Ia tidak menangkap sinonim, parafrasa, atau perbedaan konteks.

Word embedding menjawab sebagian keterbatasan tersebut. Dengan memetakan kata ke vektor padat berdasarkan konteks, word embedding membuat kata-kata yang bermakna dekat memiliki vektor yang berdekatan. Pada demo, *astronaut* berdekatan dengan *survive*, *Mars*, dan *stranded*; *love* berdekatan dengan *fall* dan *young*. Namun, karena setiap kata hanya memiliki satu vektor, embedding statis belum mampu membedakan "space exploration" dari "space ranger". Mean pooling juga menghilangkan urutan dan komposisi kalimat.

Sentence embedding melangkah lebih jauh. Dengan encoder seperti Sentence-BERT, seluruh kalimat dipetakan menjadi satu vektor yang mempertimbangkan konteks. Pada demo, Sentence-BERT menghasilkan peringkat yang lebih koheren: *The Martian*, *Gravity*, dan *Arrival* berada di atas *Toy Story*, meskipun *Toy Story* berbagi kata "space". Ini menunjukkan bahwa representasi tingkat kalimat lebih baik dalam menangkap makna, dengan konsekuensi biaya komputasi yang lebih tinggi.

Dengan demikian, tidak ada satu pendekatan yang selalu terbaik. Pilihan bergantung pada kebutuhan: TF-IDF untuk baseline dan interpretabilitas, word embedding untuk semantic generalization yang ringan, sentence embedding untuk kualitas semantik yang tinggi, dan hybrid ranking untuk produksi. Penelitian lanjutan dapat menguji pendekatan ini pada dataset berbahasa Indonesia yang lebih besar, menambahkan evaluasi pengguna, serta membandingkan model multibahasa dan model yang di-fine-tune pada domain tertentu.

Referensi

1. Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender Systems Handbook* (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6
2. Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/
3. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv:1301.3781*. https://arxiv.org/abs/1301.3781
4. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems*, 26.
5. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP 2014*, 1532–1543. https://doi.org/10.3115/v1/D14-1162
6. Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, 135–146.
7. Le, Q. V., & Mikolov, T. (2014). Distributed Representations of Sentences and Documents. *Proceedings of ICML 2014*, 1188–1196.
8. Arora, S., Liang, Y., & Ma, T. (2017). A Simple but Tough-to-Beat Baseline for Sentence Embeddings. *International Conference on Learning Representations*.
9. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992. https://doi.org/10.18653/v1/D19-1410
10. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
11. Cer, D., Yang, Y., Kong, S.-Y., Hua, N., Limtiaco, N., St. John, R., et al. (2018). Universal Sentence Encoder. *arXiv:1803.11175*. https://arxiv.org/abs/1803.11175
12. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
13. Levy, O., & Goldberg, Y. (2014). Neural Word Embedding as Implicit Matrix Factorization. *Advances in Neural Information Processing Systems*, 27.
14. Levy, O., Goldberg, Y., & Dagan, I. (2015). Improving Distributional Similarity with Lessons Learned from Word Embeddings. *Transactions of the Association for Computational Linguistics*, 3, 211–225.
15. Goldberg, Y., & Levy, O. (2014). word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method. *arXiv:1402.3722*. https://arxiv.org/abs/1402.3722
16. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8), 30–37.
17. Kunaver, M., & Požrl, T. (2017). Diversity in Recommender Systems: A Survey. *Knowledge-Based Systems*, 123, 154–162.
18. Sahlgren, M. (2005). An Introduction to Random Indexing. *Proceedings of the Methods and Applications of Semantic Indexing Workshop*.

Lampiran A: Struktur dan Cara Menjalankan Program

Program pendukung berada di folder `program/` pada repositori. Struktur utamanya:

```
content-based-filtering/
├── index.html                halaman demo interaktif
├── report.html               sumber laporan
├── report.pdf                laporan siap kumpul
├── assets/
│   ├── app.js                logika demo
│   ├── demo-data.js          data hasil program
│   ├── demo-data.json        data mentah
│   ├── style.css             tampilan demo
│   └── report.css            tampilan laporan
└── program/
    ├── cbf_embeddings.py     program utama
    ├── requirements.txt      dependensi
    ├── output.txt            output eksekusi
    └── README.txt            panduan singkat
```

A.1 Menjalankan program

Program membutuhkan Python 3 dan paket `numpy` serta `scikit-learn`. Untuk perbandingan Sentence-BERT, diperlukan paket `sentence-transformers`. Jika paket tersebut tidak tersedia, program tetap berjalan dengan TF-IDF dan mean pooling.

```
pip install numpy scikit-learn
pip install sentence-transformers   # opsional, untuk SBERT

python3 cbf_embeddings.py
```

Program akan menulis ringkasan ke terminal dan ke `output.txt`, serta menghasilkan `assets/demo-data.json` dan `assets/demo-data.js` untuk halaman demo. Halaman demo dapat dibuka langsung dari `index.html` atau melalui GitHub Pages.

A.2 Fungsi utama

Fungsi	Peran
`tokenize()`	Tokenisasi, stopword removal, normalisasi bentuk jamak.
`build_word_vectors()`	Co-occurrence, PPMI, SVD, normalisasi L2.
`mean_sentence_vector()`	Mean pooling vektor kata menjadi vektor kalimat.
`tfidf_vectors()`	Baseline TF-IDF dengan normalisasi L2.
`sbert_vectors()`	Sentence embedding dengan Sentence-BERT, opsional.
`rank_items()`	Peringkat item berdasarkan cosine similarity.
`cosine_breakdown()`	Rincian dot product, norma, dan cosine.

Tabel 6. Fungsi utama pada `cbf_embeddings.py`.

A.3 Catatan reproduksibilitas

Hasil dapat berbeda sedikit jika versi `scikit-learn`, `sentence-transformers`, atau model berubah. Untuk hasil yang persis sama dengan laporan, gunakan data dan model yang sama (`all-MiniLM-L6-v2`) serta jalankan program pada lingkungan Python 3.12. Program menggunakan `random_state=42` pada SVD.

Lampiran B: Ringkasan Output Program

Berikut ringkasan output `cbf_embeddings.py` pada data demo. Output lengkap tersedia di `program/output.txt`.

```
CONTENT-BASED FILTERING DENGAN WORD & SENTENCE EMBEDDING
Jumlah item            : 12 film
Jumlah kata deskripsi  : 112 token

[1] WORD EMBEDDING: co-occurrence -> PPMI -> SVD
    Ukuran kosakata        : 92
    Matriks co-occurrence  : 92 x 92
    Dimensi vektor         : 48
    Explained variance SVD : 89.20%
    Sel PPMI tak nol       : 366

    Tetangga terdekat:
      space      -> shuttle (0.56), destroy (0.42), ranger (0.34), orbit (0.34)
      astronaut  -> survive (0.45), fight (0.45), mars (0.43), stranded (0.43)
      love       -> fall (0.67), young (0.58), test (0.38), artist (0.37)
      crime      -> patriarch (0.75), aging (0.52), control (0.49), family (0.42)

[2] SENTENCE EMBEDDING
    Mean pooling           : rata-rata vektor kata, lalu normalisasi L2
    TF-IDF baseline        : 92 dimensi kosakata
    Sentence-BERT          : all-MiniLM-L6-v2 (384 dimensi)

[3] REKOMENDASI UNTUK QUERY: Interstellar
    TF-IDF
      1. The Martian          0.1743
      2. Gravity              0.1417
      3. Toy Story            0.1414
    Word Embedding + Mean Pooling
      1. The Martian          0.5225
      2. Gravity              0.4860
      3. Toy Story            0.3690
    Sentence-BERT
      1. The Martian          0.4770
      2. Gravity              0.3277
      3. Arrival              0.3219

[4] CONTOH PERHITUNGAN COSINE (Word Embedding + Mean Pooling)
    Pasangan: Interstellar vs The Martian
    dot(q, d)      = 0.1790
    ||q||          = 0.5397
    ||d||          = 0.6348
    cosine         = 0.5225
```

Output di atas diringkas; angka dapat berubah sedikit jika lingkungan atau model berbeda. File lengkap tetap disertakan dalam repositori.

> **Akses demo.** Halaman interaktif: https://najibrobbani74.github.io/college-task/content-based-filtering/. Laporan PDF, kode Python, dan data demo tersedia di halaman yang sama.